Team 57. Team Members: Josh Childs (1549150), Enzo Noel (1675346), Molly Stow (1356089), YU-WEI LIN (1537312)

Video:

GitLab Repo: `https://gitlab.unimelb.edu.au/JCCHIL/comp90024_team_57`

System Architecture

Overview

The system is designed to gather data from three different platforms: Bluesky, OpenAustralia and Reddit. The data is processed by fission functions and stored in elastic search indices. Analysis is performed by seperate fission functions, and the result of any analysis is cached in elastic search for future requests. User facing functions aggregate results from the analysis functions, which is displayed to the user in a Jupyter Notebook. The system is designed to be modular and scalable, so that we can easily add new platforms or data sources in the future and handle varying amounts of data. A high level overview of the architecture is show in Figure **??**.

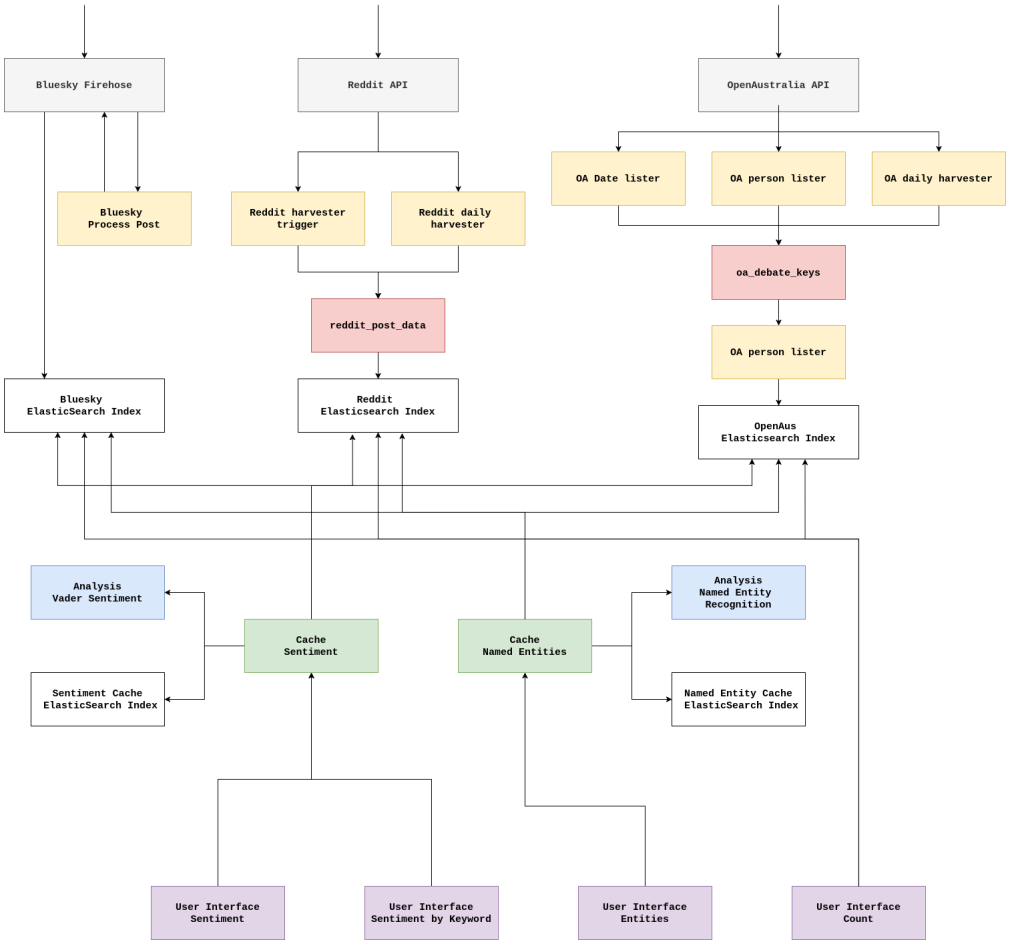

Figure 1: System Architecture. White rectangles are Elastic Search Indices, Grey rectangles are external data sources and colored rectangles are fission functions.

Bluesky

Bluesky has a firehose which gives access to all changes on the bluesky network. To handle the data from the firehose, we split the processing into two parts. The first part is receiving data from the firehose, which is written in Go and containerized in a docker image. The docker image is deployed in to kubernetes, with a single replica so kubernetes will restart the container if an error occurs. The second part is decoding and extracting the useful information from a repository commit. The AT protocol sends data in blobs, which needs to be decoded. This is handled by a fission function, which handles POST requests containing the repository commit in the body, and returns a JSON of the data we want.

The container begins by opening a websocket to the firehose endpoint `com.atproto.sync.subscribeRepos`. When we receive a `RepoCommit`, we call the fission function `processPost` which handles decoding the message. A commit could be a like on a post, a new post, a repost, or some other type of update. `processPost` returns either a 200 with a JSON of the post which contains the `cid`, `did`, `text` and `createdAt` fields, or a 204 if the commit was not a post. Bluesky uses `cid` as the unique identifier of a commit, so we reuse this in the elastic search index as the id. `did` is an identifier for the author of the post, `text` is the body of the post and `createdAt` is the post timestamp. The response is indexed to ElasticSearch using the bulk indexer.

We index the documents to Elastic Search from the firehose container. We could have the fission function index documents to ElasticSearch, but then each function would need to obtain a client login for each document, and index each document individually. In this case, the volumne of requests results in the ElasticSearch instance becoming overwhelmed by the number of requests, and would stop accepting requests. Using the bulk indexer for ElasticSearch reduces indexing overhead and increases speed.

Open Australia

The OpenAustralia (OA) API provide a REST API which we can use to query the Australian parliament's Hansard reports, and also some user comments.
A redis message queue is used to handle the data inflow from the OpenAustralia API. The queues are created by sending a post request to the `enqueue` function, which allows messages to be stored in "topic" queues.
The general structure of the OpenAustralia data inflow is as follows:

1. An "entrypoint" fission function is called, which adds the parameters (keys) of the request to the redis message ueue `oa_debate_keys`.

   ```
   {
       "house": "senate" or "representatives",
       "date" or "person": date in format YYYY-MM-DD or person id
   }
   ```

2. The `oa-debate-harvester-by-details` fission function is triggered by this redis message queue.

   This function queries the OpenAustralia API for the debate details, and then queries the OpenAustralia API for the comments on that debate. This raw data is then stored in another redis queue `oa_debates_data`.

3. The `debate-adder` fission function is triggered by this redis message queue. The data is then processed and stored in the elastic search index `oa-debates-comments`.

   A parent-child relationship is created between the debate and the comments, so that we can query the comments for a specific debate. And also between debates and their parent topics, so that we can query the debates for a specific topic.

There are three main entrypoints to gathering OpenAustralia data.

- `oa-date-lister` - Queries the OpenAustralia API for dates in a year where debates were held.

  This uses a httptrigger route `oa-dates-with-debates`, providing the year as a parameter.

- `oa-person-lister` - Queries the OpenAustralia API for all debates held by a specific person.

  This uses a httptrigger route `oa-people-year-house`, providing the year and house (senate or represenatives) as parameters.

- `oa-daily-debate-harvester` - Queries the OpenAustralia API for all debates held on a specific date, and then queues the results for processing.

  This is accessed through a time trigger, which runs once a day.

The first two entrypoints are called manually to scrape past data from the backlog, while the third is called daily to gather new data. All of the functions needed to gather and process OpenAustralia data are contained within the `oa-debates` fission package.

0.1 Reddit Data Processing Architecture

Data harvesting and processing architecture for Reddit is designed to handle both ad-hoc, parameterized scraping of Reddit data and a regular daily collection of recent posts. The system uses a Fission serverless workflow and Redis message queues to orchestrate smooth data flow from the Reddit API to Elasticsearch. The Reddit data pipeline, among other things, consists of the following major Fission functions and components:

1. **Entrypoint Functions (Initiating Data Collection):** There are two main Fission functions that act as entrypoints for Reddit data collection:

   - **HTTP Triggered Harvester (`reddit-harvester-new-r`):** This function gets started via an HTTP POST request through the route `/reddit-new/scrape/`. It is defined by the `reddit_harvester.py:main` entrypoint. The function expects a JSON payload specifying:
     - `keywords`: A list of keywords to search for.
     - `subreddits`: A list of subreddit names.
     - `limit` (optional, default: 10): The maximum number of posts to retrieve per keyword/subreddit.
     - `scrape_from` (optional, default: "2000-01-01"): A start date (YYYY-MM-DD format) for fetching posts created after this date.
     - `scrape_until` (optional, default: current date): An end date (YYYY-MM-DD format) for fetching posts created up to this date.
     - `sort` (optional): The sorting method for Reddit search results (e.g., "new", "top", "relevance").

     The `reddit_harvester.py` module uses the PRAW library to query the Reddit API and search for posts matching the supplied criteria. It also extracts top-level comments for each of the retrieved posts using a helper function named `get_post_comments` The posts and their corresponding comments are then packaged as JSON objects. These JSON objects, representing a list of posts and comments, are added to a Redis message queue called `reddit_posts_queue` using the utility function `util.py:enqueue_data`, which performs the enqueue operation via an HTTP POST request to a generic Fission `enqueue` function.

   - **Timer Triggered Daily Harvester (`reddit-daily-job`):** This function is automatically executed once a day, triggered by a Fission timer configured with a `@daily` cron schedule. It initiates the `reddit_daily_harvester.py:main` entrypoint. The script searches for posts through the Reddit API by referencing a hardcoded list of subreddits (`DAILY_SUBREDDITS`) and keywords (`DAILY_KEYWORDS`) within the script. It focuses on posts created within a specific recent period defined by `DAILY_SCRAPE_DAYS_AGO` (e.g., 1 day by default, but configurable as per the script's constants). The constants `DAILY_LIMIT_PER_KEYWORD_SUBREDDIT` and `DAILY_SORT_METHOD` are also mutable. It operates in a manner similar to the HTTP harvester, retrieving posts (and possibly their top-level comments via `get_post_comments_for_daily`) (depending on some configuration there). The JSON-formatted data is then sent to this reddit$_{p}osts_{q}ueuethrough$`reddit_posts_queue`$using$`util.py:`

2. **Data Queuing (Redis):** Both harvesting functions output their collected and formatted data (typically JSON strings representing batches of posts/comments) to a central Redis list acting as a message queue named `reddit_posts_queue`. Address for the Redis instance used by the Fission Message Queue Triggers is: `redis-headless.redis.svc.cluster.local:6379`.

3. **Data Indexing into Elasticsearch (Adder Function):** The `reddit-to-es-new-r` Fission function, with its entrypoint at `reddit_to_elasticsearch.py:main`, is responsible for consuming data from the `reddit_posts_queue`. There is a Fission Message Queue Trigger (MQT) of `mqtkind keda` watching `reddit_posts_queue`. When new messages (batches of posts/comments) get enqueued, KEDA starts up `reddit-to-es-new-r`. The function loops over posts or comments in the passed batch. For each item, it checks for any document with the same `post_id` (acting as the unique identifier for both posts and comments) already present within the `reddit` index in Elasticsearch, to avoid duplicates. If the item does not exist within this index, it gets indexed within the `reddit` index. The mapping for the index is contained within `reddit-index.json`. If processing of the message fails after the configured number of retries (say 3 retries), the troublesome message is put onto an error topic/list in Redis named `reddit_adder_error` for further analysis and debugging.

This multi-tier, queue-based architecture allows maximized decoupling of data harvesting and data indexing, with resilience and scalability features. All Fission functions for the Reddit pipeline reside within the `reddit-harvester-new-r` Fission package and use the `shared-data` ConfigMap for their configurations (such as API keys, Elasticsearch connection details, and so on).

Sentiment Analysis

Choosing an Analysis Method In order for us to perform sentiment analysis on the posts from the various social media platforms we whose, we had to choose between different avaiable options. However, since we didn't want to reinvent the wheel and risk having analysis of lesser quality we decided to use a pre-existing library. Now the choice of library came down to ease of use and overall consistency with our project.
This is why we decided to use the `VADER` library. VADER is a lexicon and rule-based sentiment analysis tool that is specifically tuned to sentiment analysis in social media posts and text. It is capable of detecting sentiment in text, and it is particularly effective for short texts such as tweets or posts.

Putting it into Practice For our project, we decided to keep the sentiment analysis simple and only use a couple of values from the vader sentiment analysis function which are the following: positive, negative and neutral.
The next step was of course to implement the analysis function in our code and make it available for other functions to use. This was done by creating a new fission function called `sentiment-analysis` which takes in a string and returns the sentiment analysis values. We had to manage the dependencies and make sure that the fission function had access to the `VADER` library. This was done by creating a fission package.

Named Entity Analysis

Analysis

Performing the analysis described above takes a non trivial amount of time. 3 simple options come to mind to handle this.

- Calculating the analysis results for all posts. Given the large number of posts we have in some indexes, bluesky in particular, this would be infeasible.
- Run the analysis on the posts queried by the user. This may take a long time if the query contains a large number of posts.
- Setup a fission function to periodically calculate and store the values for a subset of posts. This would be a better option than the two previous ones, but requires foresight into what posts are relevant, or increased effort in changing the focus.

Given the disadvantages of each, instead we implement a hybrid of 2 and 3. We calculate the values for posts queried by the user, and cache these in a separate elastic search index. When the user queries those posts again, we pull values from the cache index instead of using the analysis functions. This process is described in Algorithm **??**. To increase throughput, we send the cache function a list of post id's to retrieve, reducing the overhead of authenticating elastic search clients.

Algorithm 1 Caching Analysis Results

```
Ids ← [id_0, ..., id_n]
Index ← index                                   ▷ ElasticSearch index containing the id's
Field ← field                                   ▷ Field which is being analysised
CacheEntry ← queryESForIds(Ids, cacheName, Index)   ▷ Fetch any entries from the elastic search
                                                    cache index which match the index and
                                                    any id in Ids
for id in Ids do
    if id not in CacheEntry then
        text ← queryES(Index, id, Field)        ▷ Query Index for the document with id id
                                                  and retrieve the field Field
        value ← queryAnalysisFunction(text)     ▷ Compute the value to cache
        INSERTES(cacheName, Index, id, value)   ▷ Insert the value to the cache index
        APPEND(CacheEntry, {id, value})
    end if
end for
return CacheEntry
```

User Interface

The user interface consists of Jupyter Notebook, and fission functions to collect the data. We have 4 fission functions,

- `/ui/sentiment/keyword/:keyword/start/:start/end/:end` Aggregates the sentiment per day from the start date to the end date, filtering posts by a keyword.
- `/ui/sentiment-averager/type/:type` Calculates the average sentiment for posts containing a keyword.
- `/ui/counts/start/:start/keyword/:keyword` Calculates the number of posts containing a keyword, per day since the given start date.
- `/ui/named-entities/label/:label` Finds the named entities belonging to a given label.

Each function queries the relevant fission functions, or ElasticSearch, and collects the result into a form ready to be displayed to the user. We use MatPlotlib for most of the visualisations, and the WordCloud python package for creating word clouds of named entities.

Scenarios

Overview Today, it is nearly impossible to spend time online without witnessing a discussion about politics. For many people, this is not a topic that is discussed politely, calmly and with respect. A thread on a hot topic quickly can become intense, escalating into arguments and attacks.
But is it truly like this everywhere, or are we just seeing the worst side of the internet?
Are there still sides of social media where people present their political opinions in a pleasant and happy way? And where does all this anger come from? Are Australians egged on by high valence political speeches? Do some politicians and parties use more aggressive language, and does this align with the discussions they spark?

In this project, we set out to answer these questions by analysing political discussions held on two different platforms: Bluesky and Reddit. We thought it would be interesting to compare the sentiment and topic of these discussions to those of political speeches being held in parliament, as provided through OpenAustralia.

Analysis Goals

We had three main goals for our analysis.

Comparing the topics We first wanted to see what political topics were popular for discussion across platforms. Do users of different social media platforms have different priorities? Do they discuss the same topics, and do these align with what is being discussed in parliament? We investigated topics such as "housing", and entities, including politicians such as "Pauline Hansen", Australian locations, and parties, such as "Labor".

Comparing the overall sentiment We aimed to contrast the overall sentiment of these discussions and debates over time. We wanted to see if the conversations held on some of these platforms were more positive or negative than others, and whether there were any clear trends.

Comparing the sentiment by topic We also wanted to see if the sentiment of these discussions differed depending on the topic being discussed. For example, do users of Reddit post more positively about the topic of housing than those on BlueSky?

Comparing the topics over time We then wanted to contrast the sentiment and topics of these social media discussions to the actual debates being held. This allowed us to see if topics were discussed immediately after a relevant debate, and whether an intense debate correlated with a shift in sentiment online.

Fission

Fission Specs

In alignment with the principle of declarative application management, we used fission specs to define the desired state of our fission setup.
This allows us to easily manage the state of our system, add new functions and edit the configuration of existing functions. It also means that our infastructure is "self healing", as fission will automatically re-deploy any functions which are not in the desired state. This was espeically important when working in a shared environment with a large number of group members.

NeCTAR Research Cloud

Platform Discussion

Benefits A benefit of the NeCTAR Research Cloud and fission is the ability to use multiple programming languages. Due to issues with python, we switched the Bluesky harvester to Golang. Additionally the analysis caching was written in Go, the main benefit of using Go is the easy concurrency for requesting new analysis results. Team members not familiar with Go could still use the cache function since it could be accessed as a fission function from Python code.

Challenges A major challenge our group faced was adapting to the shared nature of the NeCTAR research cloud. At several points during the project, we encountered issues arising from multiple people attempting to modify the system at once.
Some of the minor issues included conflicting fission deployments, or source code being removed without the fission yaml specs being removed, resulting in a spec that could not be deployed. However, we also encountered more serious issues. Near the beginning of the project, the cluster was deleted and the setup

was unexpectedly re-run, meaning that we had to re-build our software stack. And not long after the cluster was re-setup, a duplicate ElasticSearch deployment was made to the default namespace, alongside the ElasticSearch deployment in the elastic namespace. This didn't seem to break anything and was removed without affecting the project. A challenge of NeCTAR is the lack of Role Based Access Control within a group, meaning we couldn't have one person responsible for managing the cluster.
Even at a small group scale, it was difficult to coordinate the changes being made to the system. This was mitigated by the use of fission specs, as we could easily re-deploy the functions to a different worker node. These challenges highlighted the benefits of using an Infrastructure as Code approach, as we could recover from these setbacks.
Another challenge of using fission is the inability to carefully instrument programs when issues occur within a function. In this case, Bluesky would not work in python. A simple function to pull from a feed would work locally but not in fission. Tracing it back to a minimal example, having `import atproto` in a python file would cause the function to not execute at all. So we switched to using Go for the Bluesky handling.

Application Scaling

Bluesky To handle processing the large number of posts in the bluesky firehose, we utilise a new deploy pool of fission functions. We set the minimum number of pods to 1, and the maximum to 30. This allows fission to increase or decrease the number of pods, up to 30, depending on the current load. Since the firehose container is implemented using Go, we spawn a Go thread for each request, allowing the firehose to scale vertically to meet demand since we can't scale horizontally.

Using grafana, we can monitor the system resources used by the bluesky firehose. At the peak, the fission function handles 35,000 requests per second, with around 5,000 of those requests containing a post, shown in Figure **??**. The firehose websocket pod utilises 0.8 cpu's at the peak, and 0.4 cpu's for the fission function, shown in Figure **??**.

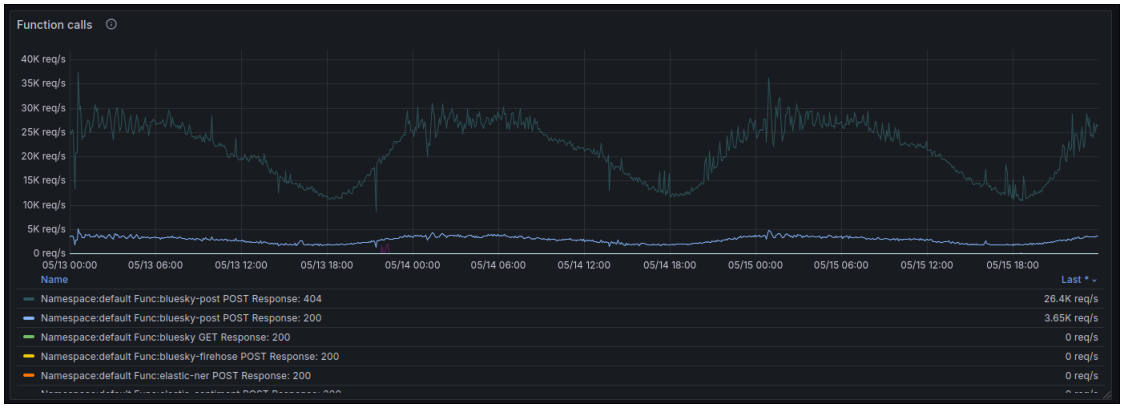

Figure 2: Bluesky fission function calls.

OpenAustralia The data influx of the OpenAustralia api was of a less intense "velocity" compared to that of BlueSky and Reddit. This meant that the regular scraping could be done on a daily basis, gathering data from debates held two days prior, with to allow complete collection of data, as the Hansard reports are finalised by 5pm the day after the debate. This was achieved using a time based fission trigger `oa-daily-debate-harvester`, which runs once a day. Similarly to the BlueSky setup, the number of pods were dynamically allocated as needed. However, due to the lower amount of incoming data the minimum number of pods was set to 0, as the resources required when starting a new pod would not significantly affect the performance.
The message queues used a Keda workload autoscaler. This monitored the number of messages in the queue, and scaled the number of pods up or down depending on the number of pods required. This helped handle

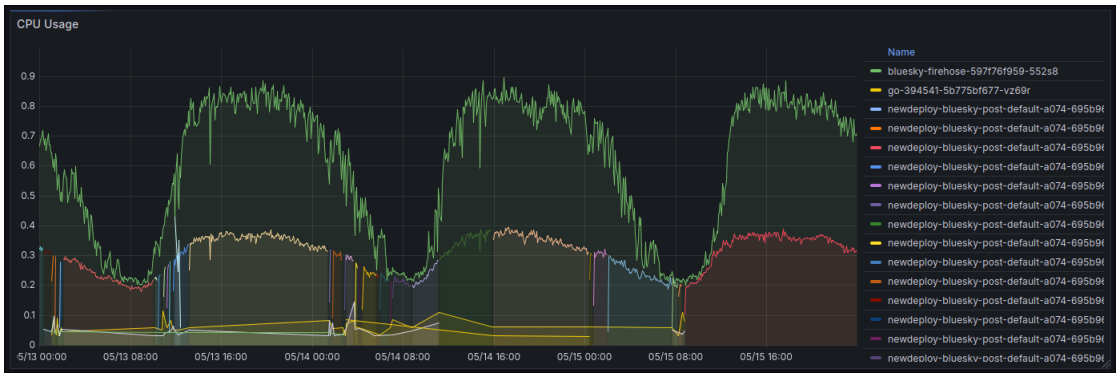

Figure 3: Bluesky cpu usage.

the variabible load of the OpenAustralia API, as the number of messages in the queue could vary from 0 to 1000+, especially when gathering data from previous years.
As a side comment, the usefulness of the `oa-daily-debate-harvester` was limited while collecting data for our project, as The Houses of Parliament are in recess from 16 May until 25 May 2025 and so no new debates were available to add.

Error Handling

Bluesky The bluesky websocket runs inside a container, instead of a fission function, since the websocket can listen indefinitely. This container is deployed to kubernetes with one replica. This means the container will be re-deployed to another worker node if one fails, and kubernetes will spin up another replica if the current one crashes from an error.

OpenAustralia A modular, "Event Driven Architecture" approach was taken to handle errors in the OpenAustralia API. The logic associated with querying the OpenAustralia API was split into three key parts: the entrypoint functions, the harvester, and the adder.
These are connected by redis message queues, each with their own "error topics". This seperation of functionality improves the robustness of the system, as we can see exactly where any errors occurred, and retry only that part of the process. Somethiungsomething Event driven architecture It also means that if one part of the process fails, the outputs of the other functions will still be saved, allowing the process to continue without data loss.
This was particularly useful when querying the OpenAustralia API, as the formatting of responses is often unpredictable, even within the same endpoints.
The immediate httptrigger entrypoint functions do not have any retry logic, as they are only called once manually, and the data is stored in the redis queue. The `oa-debate-harvester-by-details` and `debate-adder` functions are triggered by the redis queues, and have retry logic built in. Both of these functions will retry up to 3 times if they encounter an error, and will log the error message to their respective "error topic" redis queue.
This strategy is certainly not perfect, as it does not handle all possible errors, but it does allow us to see where the errors are occurring, and retry the failed functions.

Reddit Similarly to the OpenAustralia API, the reddit harvester is split into three parts: the entrypoint, the harvester and the adder. This provided all the same benefits as the OpenAustralia API, including seperation of functionality, and the ability to retry failed function and greater insight into where errors are occuring.

Integration Testing

Analysis

Comparing the topics: Topic visualisation through word clouds

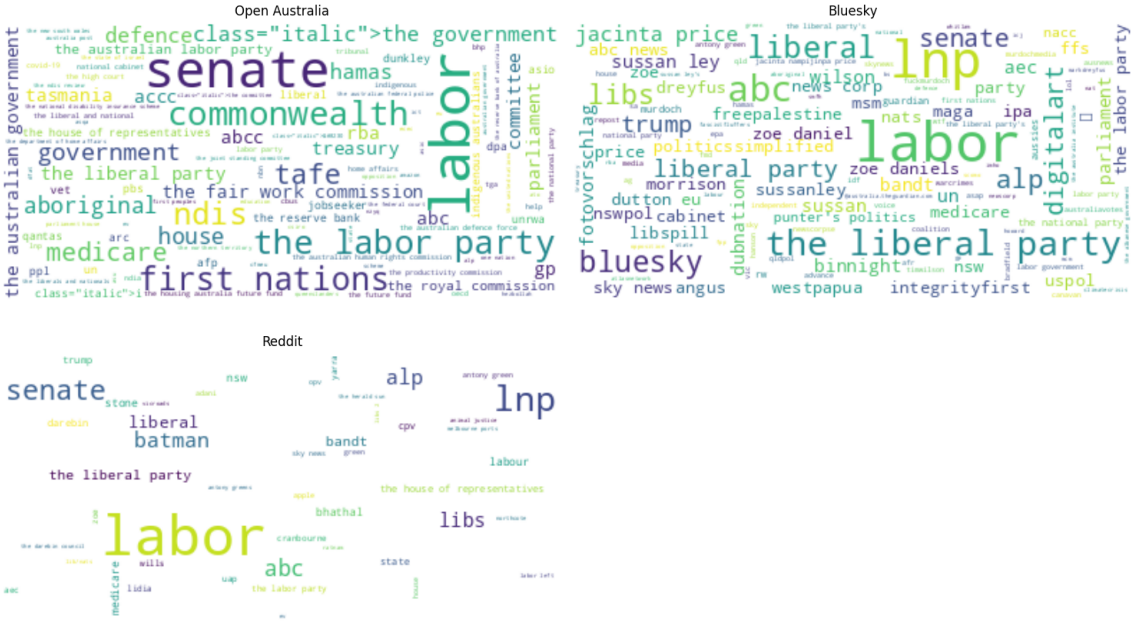

Figure 4: Word cloud of the most common "Organisation" entities in the posts and debates.

Whatever we found here. btw will replace with final images To visually compare the topics of the posts and debates, we word clouds to show the most common entities in the posts and debates. Analysis was performed using the `spacy` library, and the word cloud was generated using the `wordcloud` library.

Comparing the topics over time:

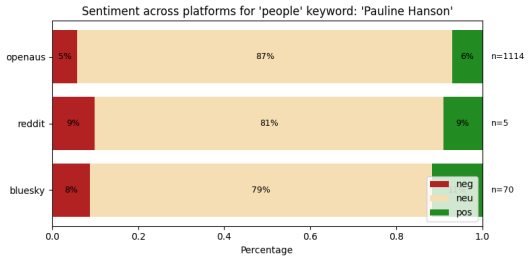

Figure 5: Comparison of sentiment for the keyword "Pauline Hanson".

The number of posts and debates containing a specific set of keywords was calculated over time. This allowed us to see if the topics of the posts and debates were similar, and if they changed over time. We hoped to see if the topics of online discussions changed after a debate was held to determine patterns of possible influence.

Comparing the overall sentiment: Stacked bar chart over time

The overall sentiment of the posts and debates was calculated using the `VADER` library. idk

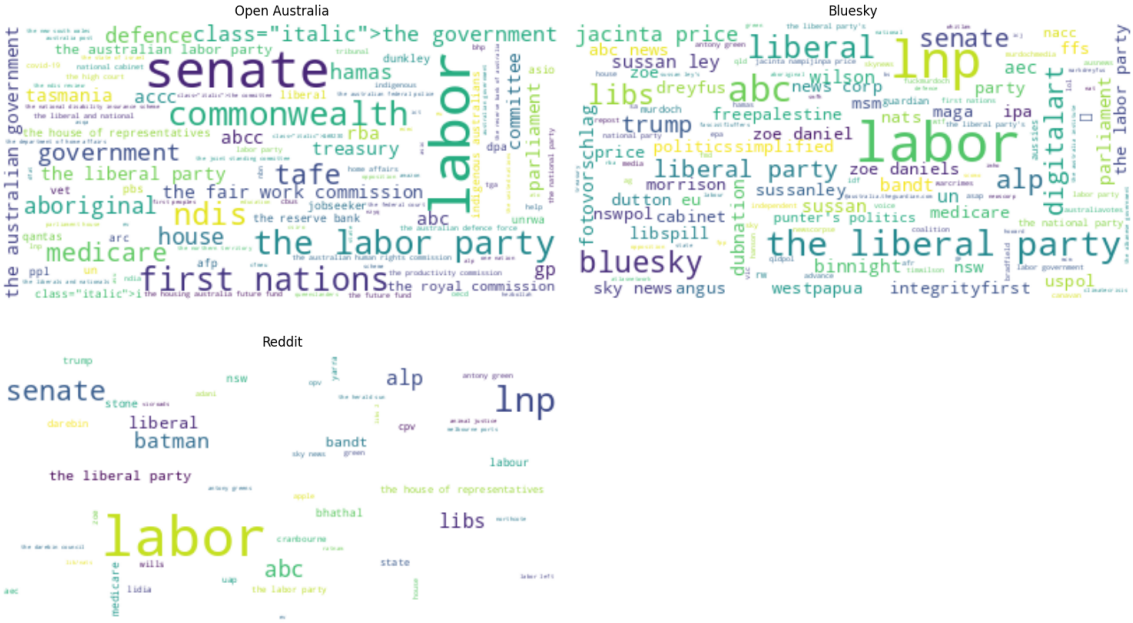

Figure 6: Overall sentiment over time from TIME to TIME.

Comparing the sentiment by keyword: idk what to call this one

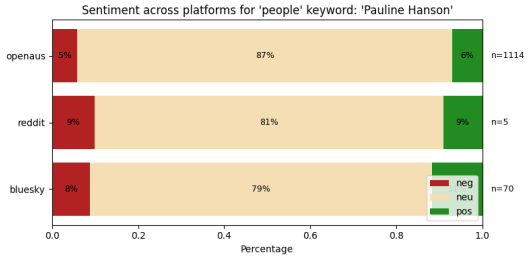

Figure 7: Comparison of sentiment for the keyword “Pauline Hanson”.

A small number of topics were selected based on their prevalence in the posts and debates, and their relevance to the current political climate.